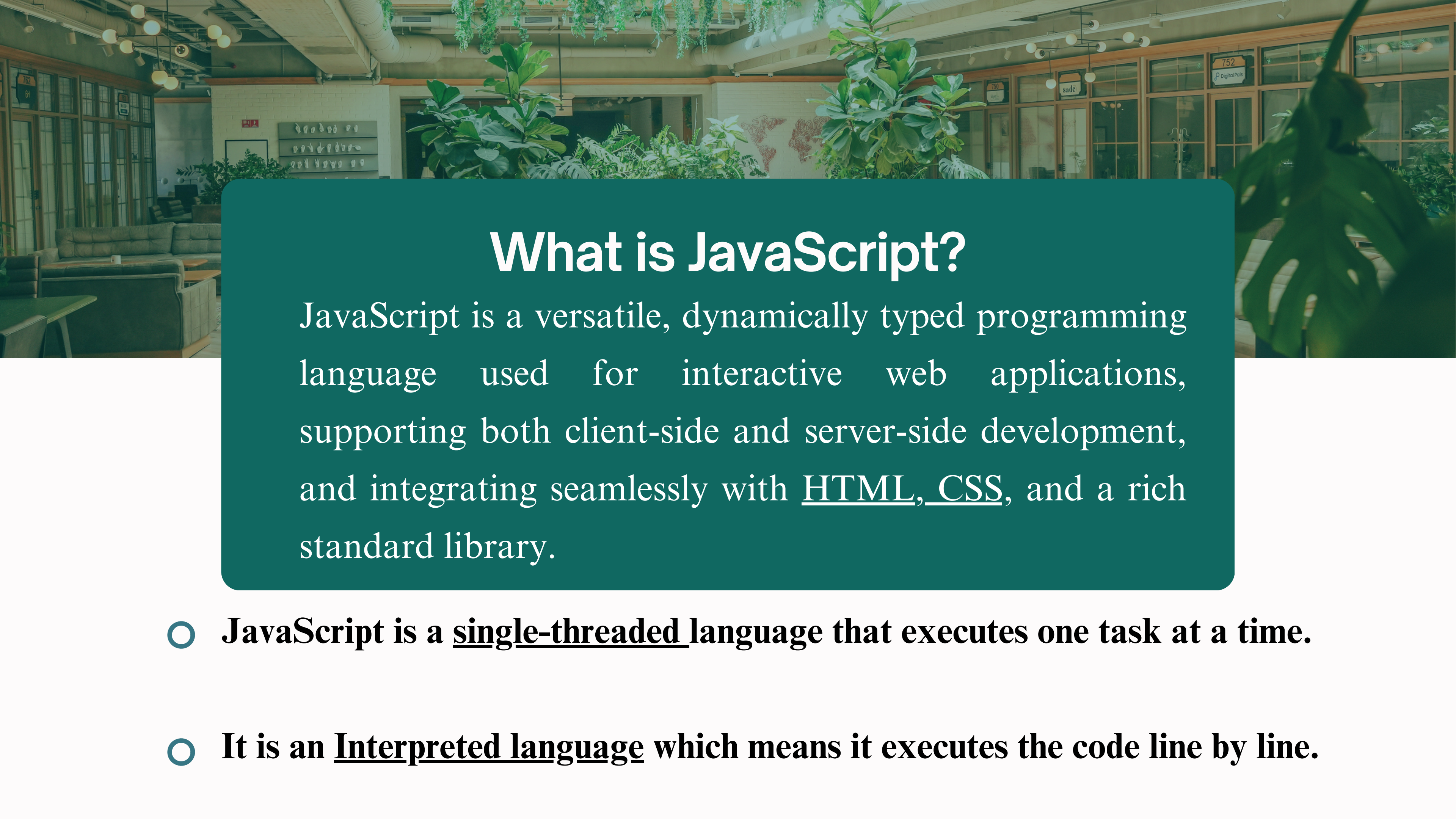


JavaScript



- What is JavaScript?
- Overview of JavaScript and its uses
- Where to run the JavaScript programs
- Basic syntax
- Data types
- Variables and their scope (let, const)
- Operators and expressions
- Control Statements
- Object creation
- Arrays
- Functions and their importance in JavaScript
- Function declaration and expression
- Function scope



A modern office interior with large windows, indoor plants, and a teal overlay containing text.

What is JavaScript?

JavaScript is a versatile, dynamically typed programming language used for interactive web applications, supporting both client-side and server-side development, and integrating seamlessly with HTML, CSS, and a rich standard library.

- JavaScript is a single-threaded language that executes one task at a time.
- It is an Interpreted language which means it executes the code line by line.

Overview of Javascript and its uses

- Origin
 - Difference between Java and JavaScript
 - Uses
 - Two ways to embed JavaScript in an HTML document
-

Origin

JavaScript, which was originally developed by **Netscape**, was initially named **Mocha** but soon after was renamed **LiveScript**. In late 1995 LiveScript became a **joint venture of Netscape and Sun Microsystems**, and its name again was changed to **JavaScript**.

A language **standard** for JavaScript was developed by the **European Computer Manufacturers Association (ECMA)** as **ECMA-262**. This standard has also been approved by the **International Standards Organization (ISO)** as **ISO-16262**.

Difference between Java and JavaScript

Feature	Java	JavaScript
Type	Object-Oriented Programming Language	Scripting Language (primarily for the web)
Platform	Runs on JVM (Java Virtual Machine)	Runs in browsers or with Node.js
Compilation	Compiled into bytecode (.class files)	Interpreted
Use Cases	Enterprise apps, Android apps, server-side apps	Web pages, client-side interactivity, web apps
Object-Oriented	Fully object-oriented	Prototype-based (not class-based originally)
File Extension	.java	.js
Complexity	More complex, suitable for large applications	Simpler, easier to start with for beginners

Uses

Used in:	Examples	By Using:
Web Development (Client-Side)	Form validation, Imagesliders and carousels, Real-time updates	JavaScript
Web Development (Server-Side)	Create web servers and APIs, Connect to databases, Handle user authentication	Node.js
Mobile App Development	Android/iOS mobile apps	React Native, Ionic, NativeScript
Desktop App Development	Visual Studio Code, Slack (desktop version), Discord (desktop version)	Electron.js
Game Development	2D/3D games.	Phaser, Three.js (for 3D), Babylon.js
Data Visualization & Dashboards	charts and graphs in web apps.	Chart.js, D3.js, Highcharts, Google Charts
Automation & Scripting	File management, Automated testing (e.g., Selenium, Puppeteer), Web scraping	JavaScript with Node.js or browser scripting

Two ways to embed JavaScript in an HTML document

There are two different ways to embed JavaScript in an HTML document:

- *implicitly*
- *explicitly*

In **explicit embedding**, the JavaScript code **physically resides** in the HTML document.

In **implicit embedding**, the JavaScript can be placed in its **own file**, separate from the HTML document.

Implicit embedding, has the advantage of **hiding the script** from the browser user.

When JavaScript scripts are explicitly embedded, they can appear in either part of an HTML document—**the head or the body**—**depending on the purpose of the script**.

Where to run the JavaScript programs

Platform	Examples
Web Browsers	Chrome, Firefox, Safari
Servers	Node.js
Terminals	Using node
Mobile Apps	React Native, Ionic
Desktop Apps	Electron.js(VS Code, Slack), NW.js
Online Sandboxes	JSFiddle, CodePen, Replit
IoT / Embedded Devices	Espruino, Tessel

IDE's

Name	Type
Visual Studio Code (VS Code)	Code Editor (Lightweight IDE)
WebStorm	Full-featured IDE
Sublime Text	Code Editor
Atom (now sunsetted)	Code Editor
Brackets	Code Editor
Replit	Online IDE
CodeSandbox	Online IDE

Basic Syntax

- The `<script>` tag is used to include JavaScript code inside an HTML document.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>First js program</title>
  <script>
    document.write("This writes to the webpage");
  </script>
</head>
<body>
  <h1>SSIT</h1>
  <p> i am pooja</p>
</body>
</html>
```



This writes to the webpage

To print the content

SSIT

i am pooja

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>First js program</title>
</head>
<body>
  <script>
    //Single comment line
    /*
      Multiple
      Comment line*/
    console.log("This goes to the console,I am here");
    alert("This shows a popup");
    document.write("This writes to the webpage");
  </script>
  <h1>SSIT</h1>
</body>
</html>
```

- Pop up message
- Console.log(see in console)

Lets see how to run the JavaScript on different file (implicit embedding)

JavaScript file

```
JS s2.js
1  console.log("Welcome to javascript!!");
2  a=9;
3  b=1;
4  c=a+b;
5  console.log(c);
6  alert("Your output is displayed on console!!!");
7
```

```
<> s1.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  |   <meta charset="UTF-8">
5  |   <title>Document</title>
6  </head>
7  <body>
8  |   <script src="s2.js"></script>
9  </body>
10 </html>
```

html file

JavaScript Output

JavaScript Display Possibilities

JavaScript can "display" data in different ways:

- Writing into an HTML element, using **innerHTML** or **innerText**.
- Writing into the HTML output using **document.write()**.
- Writing into an alert box, using **window.alert()**.
- Writing into the browser console, using **console.log()**.

JavaScript Statements

- A **computer program** is a list of "instructions" to be "executed" by a computer.
- In a programming language, these **programming instructions** are called **statements**.
- A **JavaScript program** is a list of **programming statements**.
- JavaScript statements are composed of: Values, Operators, Expressions, Keywords, and Comments.

- Semicolons separate JavaScript statements.

JavaScript Keywords

JavaScript statements often start with a keyword to **identify the JavaScript action to be performed**.

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

JavaScript Identifiers / Names

Identifiers are JavaScript names.

Identifiers are used to **name variables and keywords, and functions.**

The rules for legal names are the same in most programming languages.

A JavaScript name must begin with:

- A letter (A-Z or a-z)
- A dollar sign (\$)
- Or an underscore (_)

Subsequent characters may be letters, digits, underscores, or dollar signs.

All JavaScript identifiers are **case sensitive**.

JavaScript Values

The JavaScript syntax defines two types of values:

- Fixed values
- Variable values

Fixed values are called **Literals**.

Variable values are called **Variables**.

JavaScript Literals

The two most important syntax rules for fixed values are:

- 1. Numbers are written with or without decimals
- 2. Strings are text, written within double or single quotes

Example: [Share the code.](#)



JavaScript Variables

Variables are Containers for Storing Data.

JavaScript Variables can be declared in 4 ways:

- Automatically
- Using var
- Using let
- Using const

An **equal sign** is used to assign values to variables.

Example: [Share the code.](#)

Feature	var	let	const
Reassignable	✔ Yes	✔ Yes	✗ No
Redeclarable	✔ Yes	✗ No	✗ No
Scope	✗ Function scope	✔ Block scope	✔ Block scope
Use in modern JS	✗ Avoid	✔ Recommended	✔ For constants

JavaScript Data Types

Two Main Categories of Data Types

- 1. Primitive Data Types – simple values
 - 2. Non-Primitive (Reference) Data Types – complex data structures
- A JavaScript variable can hold any type of data.
 - Note: Primitive types are immutable (they cannot be changed).
 - Objects are mutable, and stored as references in memory.

Primitive Data Types

Type	Description	Example
String	Text (inside quotes)	"Hello" or 'World'
Number	Numbers (integers or decimals)	10, 3.14
Boolean	True or False values	true, false
Undefined	A variable with no assigned value	let a; → a is undefined
Null	Empty or non-existent value	let a = null;
BigInt	Very large integers	let big = 1234567890123456789012345n;
Symbol	Unique identifiers	let id = Symbol("id");

Non-Primitive Data Types

Type	Description	Example
Object	A collection of key–value pairs	{ name: "Alice", age: 25 }
Array	A list of items	[10, 20, 30]
Function	A block of reusable code	function greet() { ... }
Date	For working with dates and times	new Date()
RegExp	Regular expressions (patterns)	/abc/

Examples: Share the code.

JavaScript Operators

Javascript operators are used to perform different types of mathematical and logical computations.

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- String Operators
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators



Arithmetic Operators

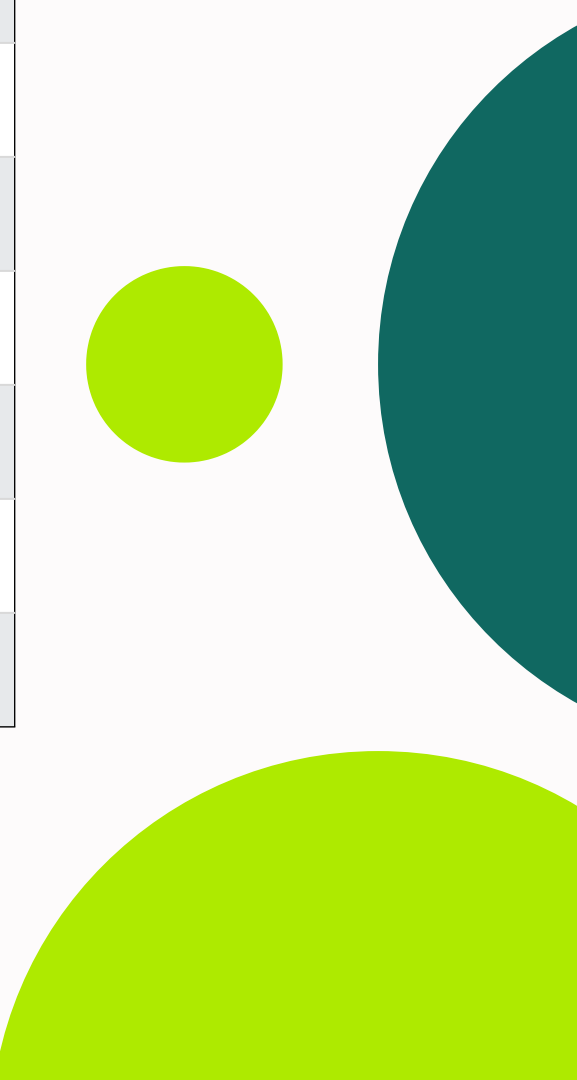
Arithmetic Operators are used to **perform arithmetic on numbers.**

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Assignment Operators

Assignment operators assign values to JavaScript variables.

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y



Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Logical Operators

Operator	Description
&&	logical and
	logical or
!	logical not

Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

● Control statements

- Conditional Statements

Types:

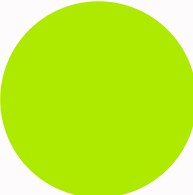
if, else, else if, switch

if statement:

Use the **if statement** to specify a block of JavaScript code to be executed if a condition is true.

Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```



else statement:

Use the **else statement** to specify a block of code to be executed if the condition is false.

Syntax:

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

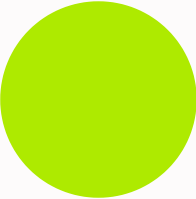
else if statement:

Use the **else if statement** to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```





switch statement:

Use the **switch statement** to select one of many code blocks to be executed.

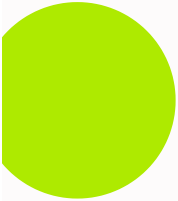
Syntax

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

- **Looping or Iterative Statements:**

Types:

for, while, do while



for loop:

The **for loop** creates a loop with 3 optional expressions:

```
for (expression 1; expression 2; expression 3) {  
    // code block to be executed  
}
```

- **Expression 1** is executed (one time) before the execution of the code block.
- **Expression 2** defines the condition for executing the code block.
- **Expression 3** is executed (every time) after the code block has been executed.

while loop:

The while loop loops through a block of code as long as a specified condition is true.

Syntax

```
while (condition) {  
    // code block to be executed  
}
```

do while loop:

The do while loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

Syntax

```
do {  
    // code block to be executed  
}  
while (condition);
```

- **Jumping Statements**

Break Statement:

The break statement terminates the current loop (for, while, do-while) or switch statement. Execution resumes at the statement immediately following the terminated block.

Continue Statement:

The continue statement skips the rest of the code within the current iteration of a loop (for, while, do-while) and proceeds to the next iteration.

● Object Creation

- An object is a special data type that allows you to store multiple related values (as key–value pairs) in a single structure.
- Object = data + methods
- It is a collection of properties that contain data.
- To create an object type we use an object constructor function.
- The **new** operator creates a blank object—that is, **one with no properties**. JavaScript objects do not have types. The constructor both creates and initializes the properties.
- The following statement creates an object that has no properties:

```
var my_object = new Object();
```

● Arrays

- An object is a special data type that allows you to store multiple related values (as key–value pairs) in a single structure.
- Object = data + methods
- It is a collection of properties that contain data.
- To create an object type we use an object constructor function.
- The **new** operator creates a blank object—that is, **one with no properties**. JavaScript objects do not have types. The constructor both creates and initializes the properties.
- The following statement creates an object that has no properties:

```
var my_object = new Object();
```

● Functions and their importance in JavaScript

A **function** is a reusable block of code designed to perform a specific task.

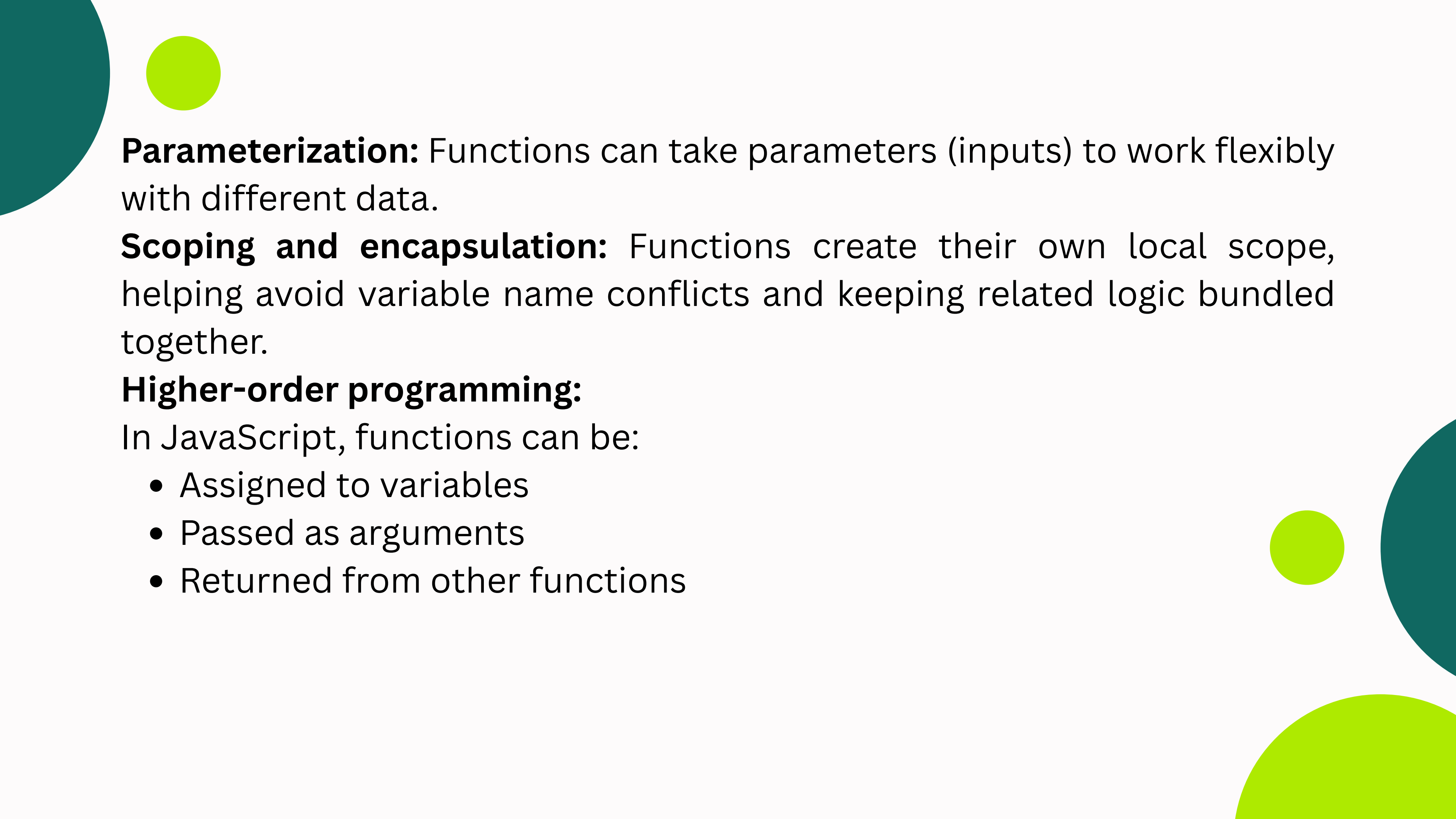
You define it once and can call (run) it whenever needed.

Why are functions important in JavaScript?

Reusability: You can avoid repeating the same code — just call the function wherever needed.

Organization and clarity: Functions help break your program into smaller, logical parts, making code easier to read and maintain.

Avoiding repetition (DRY principle): “Don’t Repeat Yourself” — functions help eliminate duplicate code.



Parameterization: Functions can take parameters (inputs) to work flexibly with different data.

Scoping and encapsulation: Functions create their own local scope, helping avoid variable name conflicts and keeping related logic bundled together.

Higher-order programming:

In JavaScript, functions can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions

● Function declaration and expression

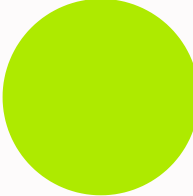
JavaScript functions are defined with the **function** keyword. You can use a **function declaration** or a **function expression**.

Function Declaration:

Syntax:

```
function functionName(parameters) {  
    // code to be executed  
}
```

Declared functions are **not** executed immediately. They are "**saved for later use**", and will be **executed later**, when they are **invoked** (called upon).



Function scope

JavaScript variables can belong to:
The **local scope** or The **global scope**

Local Variables:

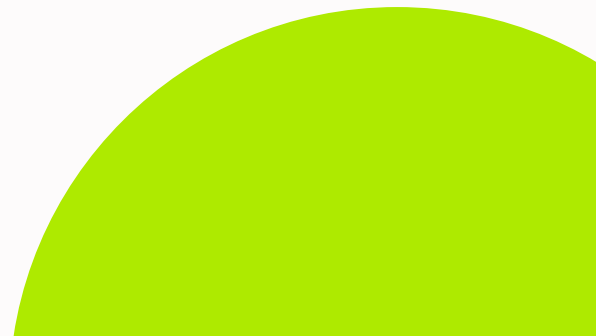
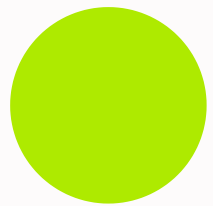
A local variable is a "**private**" variable defined **inside a function**.

A function can access all variables in the local scope.

Global Variables:

A global variable is a "**public**" variable defined **outside a function**.

A function can access all variables in the global scope:





JS

**THANK
YOU**

